KỊCH BẢN THUYẾT TRÌNH · DỰ ÁN BACKEND

Hệ thống đặt vé xem phim

Cinema Booking | FastAPI · PostgreSQL · Docker

MỤC TIÊU	Trình bày bài toán, luồng nghiệp vụ đặt vé, API và bảo mật, kiến trúc triển khai, kết quả kiểm thử tải và hướng phát triển.
THỜI LƯỢNG	Khoảng 8–9 phút, có thể rút gọn phần chạy demo nếu thời gian hạn chế.
CÁCH DÙNG	Phần trong ngoặc vuông là gợi ý thao tác hoặc chuyển slide, không cần đọc thành lời.

MỞ ĐẦU

"Kính chào thầy cô và các bạn. Hôm nay em xin đại diện nhóm giới thiệu dự án Cinema Booking - một REST API hỗ trợ đặt vé xem phim. Hệ thống được xây dựng bằng Python 3.12, FastAPI và PostgreSQL 16. Trong phần trình bày, em sẽ đi từ bài toán và luồng đặt vé đến kiến trúc, cách triển khai, kết quả kiểm thử và những giới hạn nhóm dự kiến cải thiện."

LỘ TRÌNH

01	Bài toán và vai trò người dùng	1 phút
02	Luồng giữ ghế, thanh toán và booking	1,5 phút
03	API, phân quyền và kiến trúc	2 phút
04	Công nghệ, chạy hệ thống và demo	1,5 phút
05	Kết quả kiểm thử tải	1 phút
06	Giới hạn và hướng phát triển	1 phút

PHẦN 01

Bài toán và phạm vi hệ thống

THỜI LƯỢNG ƯỚC TÍNH · 1 phút

GỢI Ý TRÌNH BÀY	Mở bằng hành trình khán giả: tìm phim → chọn suất → kiểm tra ghế.

“Bài toán nhóm hướng tới là giúp khán giả tra cứu phim, chọn suất chiếu và biết ghế nào còn trống trước khi mua vé. Người dùng đã đăng nhập có thể giữ ghế trong một khoảng thời gian để hoàn tất thanh toán. Nhân viên quản trị có thể quản lý phim và tạo suất chiếu.”

“Hệ thống hiện có ba nhóm quyền. Khách chưa đăng nhập được xem phim, suất chiếu và tình trạng ghế, đồng thời có thể đăng ký hoặc đăng nhập. Tài khoản USER được đặt, xem và hủy booking của chính mình; ngoài ra có thể giữ hoặc giải phóng ghế và thanh toán mô phỏng. ADMIN quản lý danh mục phim và tạo suất chiếu.”

“Phạm vi hiện tại tập trung vào backend API và luồng nghiệp vụ cốt lõi. Giao diện chọn vé hoàn chỉnh chưa nằm trong phiên bản này.”

PHẦN 02

Luồng đặt vé và quy tắc nghiệp vụ

THỜI LƯỢNG ƯỚC TÍNH · 1,5 phút

GỢI Ý TRÌNH BÀY	Dùng sơ đồ hoặc demo để làm rõ: hold trước, payment sau, booking được tạo cuối cùng.

“Luồng đặt vé được thiết kế theo ba bước. Đầu tiên, người dùng chọn suất và một đến năm ghế rồi gửi yêu cầu giữ ghế. Hold mặc định có hiệu lực 300 giây. Tiếp theo, chủ hold thực hiện thanh toán mô phỏng. Khi thanh toán thành công, hệ thống ghi nhận payment và tạo booking cho các ghế trong hold. Vì vậy, API booking chỉ dùng để xem và hủy; booking không được tạo trực tiếp từ yêu cầu đặt chỗ ban đầu.”

“Nếu hold hết hạn hoặc không còn ở trạng thái có thể thanh toán, yêu cầu payment bị từ chối. Người dùng chỉ có thể xem hoặc giải phóng hold thuộc về mình; booking và payment cũng được giới hạn theo chủ sở hữu. Yêu cầu thanh toán lặp lại cho hold đã thanh toán sẽ trả về kết quả cũ, giúp tránh tạo thêm giao dịch cho cùng một hold.”

“Mức tiền thanh toán demo được cấu hình ở backend bằng biến DEMO_PAYMENT_AMOUNT_VND. Đây là mô phỏng để hoàn chỉnh quy trình, chưa kết nối ngân hàng hay cổng thanh toán bên ngoài.”

PHẦN 03

API và bảo mật

THỜI LƯỢNG ƯỚC TÍNH · 1 phút

GỢI Ý TRÌNH BÀY	Có thể mở Swagger UI ở /docs và chỉ nhanh các nhóm endpoint.

“API sử dụng REST và JSON, có tài liệu tương tác tại /docs. Các nhóm endpoint chính gồm auth để đăng ký và đăng nhập; movies và showtimes để tra cứu nội dung; seats để xem tình trạng ghế; holds để giữ, xem hoặc giải phóng ghế; payments để thanh toán mô phỏng và xem lịch sử; bookings để xem hoặc hủy booking.”

“Các thao tác xem phim, suất chiếu và ghế là công khai. Tạo hoặc cập nhật phim, cũng như tạo suất chiếu, yêu cầu vai trò ADMIN. Những tài nguyên mang tính cá nhân như booking, hold và payment chỉ được truy cập bởi người sở hữu.”

“Mật khẩu được băm trước khi lưu. Người dùng gửi Bearer JWT trong header Authorization; AuthMiddleware xác thực token và nạp thông tin tài khoản, còn RoleChecker bảo vệ các thao tác quản trị. Pydantic kiểm tra dữ liệu request và lỗi nghiệp vụ được trả về dưới dạng JSON.”

PHẦN 04

Kiến trúc và công nghệ

THỜI LƯỢNG ƯỚC TÍNH · 1 phút

GỢI Ý TRÌNH BÀY	Nếu có sơ đồ kiến trúc, lần theo mũi tên API → service → repository → PostgreSQL.

“Ứng dụng được tổ chức theo các lớp API, service và repository. Request đi từ API vào service để xử lý quy tắc nghiệp vụ, sau đó repository thực hiện đọc ghi dữ liệu. SQLAlchemy bất đồng bộ giao tiếp với PostgreSQL. Các thư mục chính gồm api, services, repositories, models, schemas, core và db.”

“Các công nghệ chính là Python 3.12, FastAPI, SQLAlchemy async, PostgreSQL 16, Pydantic, JWT và Docker Compose. Cách chia lớp giúp tách xử lý HTTP khỏi nghiệp vụ và truy cập dữ liệu. Bộ kiểm thử có unit test, integration test; thư mục load_test chứa kịch bản Locust.”

PHẦN 05

Chạy hệ thống và nội dung demo

THỜI LƯỢNG ƯỚC TÍNH · 1,5 phút

GỢI Ý TRÌNH BÀY	Trong buổi trình bày, ưu tiên chạy sẵn container để dành thời gian cho luồng người dùng.

“Để khởi chạy nhanh, dự án yêu cầu Docker Desktop. Sau khi tạo file .env từ .env.example và thiết lập SECRET_KEY cùng thông tin kết nối database, chạy lệnh docker compose up -d --build. Khi API và database báo healthy, có thể mở Swagger UI tại localhost:8000/docs. Container API tự tạo các bảng còn thiếu.”

“Kịch bản demo gồm: xem danh sách phim và suất chiếu; kiểm tra ghế; đăng ký hoặc đăng nhập để lấy access token; dùng token giữ một hoặc vài ghế; thanh toán mô phỏng; rồi xem payment và booking được tạo. Cuối cùng có thể đăng nhập tài khoản ADMIN mẫu do seeder tạo để thêm phim và suất chiếu.”

“Nếu chỉ chạy API trên máy, vẫn có thể dùng PostgreSQL trong Docker và khởi động FastAPI bằng Uvicorn. Seeder có sẵn để nạp dữ liệu demo. Swagger cũng cho phép nhập Bearer token để gọi các endpoint cần xác thực.”

PHẦN 06

Kết quả kiểm thử tải

THỜI LƯỢNG ƯỚC TÍNH · 1 phút

GỢI Ý TRÌNH BÀY	Nhấn mạnh điều kiện đo: Kaggle CPU, 50 người dùng ảo, tăng 5 người/giây trong 60 giây.

“README ghi nhận một lần kiểm thử tải khi API, PostgreSQL và Locust cùng chạy trên máy Kaggle. Kịch bản sử dụng 50 người dùng ảo, tăng tải 5 người mỗi giây trong 60 giây.”

“Kết quả có 5.908 request và không ghi nhận request lỗi. Thông lượng trung bình đạt 99,39 request mỗi giây; độ trễ trung bình là 346,75 mili giây. Các mốc p50, p95 và p99 lần lượt là 290, 650 và 1.200 mili giây. CPU trung bình và cao nhất lần lượt là 49,2% và 78,2%; RAM trung bình và cao nhất là 1.231 và 1.467 MB.”

“Đây là kết quả trong cấu hình thử nghiệm đã nêu, giúp nhóm có số liệu ban đầu để đánh giá; chưa nên xem là cam kết hiệu năng cho mọi môi trường triển khai.”

PHẦN 07

Giới hạn và hướng phát triển

THỜI LƯỢNG ƯỚC TÍNH · 1 phút

GỢI Ý TRÌNH BÀY	Trình bày trung thực giới hạn hiện tại, sau đó nối sang các cải tiến cụ thể.

“Phiên bản hiện tại còn một số giới hạn. Thanh toán chỉ là mô phỏng. Chưa có giao diện đặt vé hoàn chỉnh, sơ đồ ghế, email xác nhận hoặc worker nền. Hold hết hạn được nhận biết khi API đọc hoặc ghi dữ liệu, chưa được xử lý bởi tác vụ nền.”

“Ngoài ra, việc kiểm tra ghế đang chủ yếu ở tầng ứng dụng. Khi nhiều yêu cầu đến đồng thời, hai người vẫn có thể cùng giữ một ghế; README cũng ghi rõ booking chưa có ràng buộc database để bảo đảm duy nhất ghế trong một suất. Các hướng ưu tiên là thêm ràng buộc dữ liệu chống giữ trùng, giải phóng hold hết hạn theo lịch, kết nối cổng thanh toán thật và gửi email xác nhận.”

“Nhóm cũng có thể bổ sung phân trang, làm mới và thu hồi token, mã chống gửi lặp cho yêu cầu giữ ghế, bộ nhớ đệm cho dữ liệu ít thay đổi và điều chỉnh pool kết nối theo quy mô triển khai.”

KẾT THÚC

“Trên đây là phần trình bày về hệ thống Cinema Booking. Dự án đã hoàn thiện các luồng backend chính từ tra cứu phim, giữ ghế đến thanh toán mô phỏng và tạo booking, đồng thời có kết quả kiểm thử tải ban đầu. Nhóm sẽ tiếp tục ưu tiên bảo đảm tính nhất quán khi nhiều người cùng chọn ghế và hoàn thiện các chức năng phục vụ người dùng. Em xin cảm ơn thầy cô và các bạn đã lắng nghe. Nhóm em sẵn sàng nhận câu hỏi.”

NHẮC NHANH TRƯỚC KHI TRÌNH BÀY	• Mở sẵn Swagger UI tại /docs • Chuẩn bị token USER và ADMIN từ dữ liệu mẫu • Chọn sẵn một suất chiếu còn ghế • Nêu kết quả tải kèm đúng cấu hình thử nghiệm